LeetCode 242: Valid Anagram

A comparative analysis of Frequency Array and HashMap solutions for optimized string matching

Link : https://leetcode.com/problems/valid-anagram

Solution 1: Freq Array

```
public boolean isAnagram(String s, String t){
        if(s.length() != t.length()) return false;

        int[] freq1 = new int[26];
        int[] freq2 = new int[26];

        for(char ch : s.toCharArray()){
                freq1[ch - ‘a’] += 1;
        }
        for(char ch : t.toCharArray()){
                freq2[ch - ‘a’] += 1;
        }

        return Array.equals(freq1, freq2);
}
```

- **Time Complexity:** O(N), where N is the length of the string, since we iterate through both strings of equal length once.
- **Space Complexity:** O(1) or O(K), where K is the size of the alphabet (26 for lowercase English letters), which is a constant fixed size.

Solution 2: HashMap

```
public boolean isAnagram(String s, String t){
        if(s.length() != t.length()) return false;

        Map<Character, Integer> countMap = new HashMap<>();

        for(char ch : s.toCharArray()){
                countMap.put(ch, countMap.getOrDefault(ch, 0) + 1);
        }

        for(char  ch : t.toCharArray()){
                if(!countMap.containsKey(ch) || countMap.get(ch) == 0){
                        return false;
                }
                countMap.put(ch, countMap.get(ch) - 1);
```

Harsh Kumar | 23-10-2026

```
        }
        return true;
}
```

- **Time Complexity:** O(N), where N is the length of the string, since we iterate through the strings and perform average O(1) map operations.
- **Space Complexity:** O(K) where K is the number of distinct characters in the string.

Why Solution 1 is Better

Although both solutions run in O(N) time complexity, Solution 1 (Frequency Array) is practically superior for fixed-character sets (like lowercase English letters) due to the following reasons:

- **Lower Overhead:** Primitive integer arrays avoid the object creation, boxing/unboxing, and hashing overhead associated with `HashMap`.
- **Cache Locality:** Arrays store elements sequentially in contiguous memory blocks, leading to better CPU cache utilization.

Deterministic Space & Time: Direct array indexing via `ch - 'a'` guarantees true O(1) operations, unlike map operations which can degrade due to hash collisions.